1. Best Practices & Standards 2
1.1 C# Best Practices 2
1.2 Naming Conventions - AutoQuotes 6
1.3 Naming Conventions - Data Import 7
1.4 Naming Conventions - Unified Portal 11
1.5 SQL Database Design Best Practices 13
1.5.1 1 Basics 14
1.5.2 2 Stored Procedures 14
1.5.2.1 2 a) Optimize your Stored Procedures 15
1.5.2.2 2 b) Avoid Hardcoding Values 16
1.5.2.3 2 c) Drop Your Temp Tables 17
1.5.2.4 2 d) Aliases used for fields 17
1.5.2.5 2 e) Format your code 17
1.5.2.6 2 f) Avoid using Cursors 17
1.5.2.7 2 g) Stored Procedure Template 18
1.5.2.8 2 h) Stored Procedure Recompilation 20
1.5.3 3 Tables 24
1.5.3.1 3 a) Indexing 24
1.5.3.2 3 b) Deleting Data 24
1.5.4 4 Databases 24
1.5.4.1 4 a) Creating Databases 24
1.5.4.2 4 b) Linked Servers 24
1.5.5 5 Quality Assurance 25

Best Practices & Standards

Best Practices

Best practices are designed to make your lives and your colleagues lives easier and also to ensure that your code is performing at its best.

- Click here to view the C# best practices
- Click here to view the SQL best practices

Standards & Naming Conventions

The key to a successful growing company is to ensure that whatever we build is repeatable and scalable. That it can be understood by any developer with minimal training or coaching.

In order to do that it is important that when we create objects, tables, views etc. that have the same look, feel and design.

Currently we have four environments that have similar but slight different naming conventions. Make sure when developing to adhere to the conventions or your script will be rejected by QA.

- Data Import
- AutoQuotes
- Unified Portal

C# Best Practices

> ⚠ These are not hard and fast rules that we will be measured against. Instead these best practices should be viewed as something we all strive towards to generate better, quality code.

C# Best Practices

1. Avoid putting multiple classes in a single file.
2. A single file should contribute types to only a single namespace. Avoid having multiple namespaces in the same file.
3. Avoid methods with more than 5 arguments. Use structures for passing multiple arguments.
4. Do not manually edit any machine-generated code.
    a. Use partial classes whenever possible to factor out the maintained portions.
5. Avoid comments that explain the obvious. Code should be self-explanatory. Good code with readable variable and method names should not require comments.
6. Document only operational assumptions, algorithm insights and so on.
7. Avoid method-level documentation.
    a. Use method-level comments only as tool tips for other developers.
8. With the exception of zero and one, never hard-code a numeric value; always declare a constant instead.
9. Use the `const` directive only on natural constants such as the number of days of the week.
10. Avoid using `const` on read-only variables. For that, use the `readonly` directive.

```
public class MyClass
{
 public const int DaysInWeek = 7;
 public readonly int Number;

 public MyClass(int someValue)
 {
  Number = soneValue;
 }
}
```

11. Assert every assumption. On average, every fifth line is an assertion.

```
using System.Diagnostics;

object GetObject()
{...}

object someObject = GetObject();
Debug.Assert(someObject != null);
```

12. In a `catch` statement that throws an exception, always throw the original exception (or another exception constructed from the original exception) to maintain the stack location of the original error.
13. Avoid defining custom exception classes.
14. Make only the most necessary types `public`, mark others as `internal`.
15. Always use zero based arrays.
16. Do not provide public or protected member variables. Use properties instead.
17. Avoid explicit properties that do nothing except access a member variable. Use automatic properties instead:

```
//Avoid
class MyClass
{
 int _number;

 public int Number
 {
  get
  {
   return _number;
  }
  set
  {
   _number = value;
  }
 }
}

//Correct
class MyClass
{
 public int Number
 {
  get; set;
 }
}
```

18. Avoid using the `new` inheritance qualifier. Use `override` instead.
19. Avoid explicit casting. Use the `as` operator to defensively cast to a type.

```
Dog dog = new GermanShepard();
GermanShepard shepard = dog as GermanShepard;
if (shepard != null)
{...}
```

20. Always use interfaces.
21. Avoid interfaces with one member.
22. Strive to have three to five members per interface.
23. Do not have more than 20 members per interface. Twelve is probably the practical limit.
24. Never assume a type supports an interface. Defensively query for that interface.

```
SomeType obj1;
IMyInterface obj2;

/* Some code to initialize obj1, then: */
obj2 = obj1 as IMyInterface;
if (obj2 != null)
{
 obj2.Method1();
}
else
{
 //Handle error in expected interface
}
```

25. Never hardcode strings that will be presented to end users. User resources instead.
26. Never hardcode strings that might change based on deployment, such as connection strings.
27. User `String.Empty` instead of "":

```
//Avoid
string name = "";

//Correct
string name = String.Empty;
```

28. When building a long string, use `StringBuilder`, not string.
29. Always provide a static constructor when providing static member variables.
30. Do not use late-binding invocation when early-binding is possible.
31. Use application logging and tracing.
32. Always have a `default` case in a `switch` statement that asserts.

```
int number = SomeMethod();
switch(number)
{
 case 1:
  Trace.WriteLine("Case 1:");
  break;
 case 2:
  Trace.WriteLine("Case 2:");
  break;
 default:
  Debug.Assert(false);
  break;
}
```

33. Avoid casting to and from `System.Object` in code that uses generics. Use constraints or the `as` operator instead:

```
class SomeClass
{}
//Avoid
class MyClass<T>
{
 void SomeMethod(T t)
 {
  object temp = t;
  SomeClass obj = (SomeClass)temp;
 }
}

//Correct
class MyClass<T> where T : SomeClass
{
 void SomeMethod(T t)
 {
  SomeClass obj = t;
 }
}
```

Naming Conventions - AutoQuotes

Environments	Alias	
devdb5.maintenancenet.net\aq_dev	AQSERVER	Development
devdb5.maintenancenet.net\aq_qa	AQSERVER_QA	QA
devdb5.maintenancenet.net\aq_stage	AQSERVER	Stage
appdb8.maintenancenet.net	AQSERVER	Production

Databases	Definitions
AutoQuotes	Auto Quote

Naming Conventions	
Schemas	When building a schema it must have at least one security context. The naming convection should reflect some sort of logic grouping. Do not use names like snapshot1, use snapshot (Developers only). However speak with your Team Lead before creating a new schema.
dbo	
Snapshot	They are a snapshot of the quoting infomation of what as actually processed and sent out. Read only, data in these tables are not to be reconcided.
Exclusion	

Report	
Tables	
AutoQuote[Entity]	Tables used for the AutoQuote Proces. Use Camel Case
MNet[Entity]	
SpecialDiscount[Entity]	
Indexes	TBD
[TableName][Type][Cluster]	Type: U = Unique Or N = NonUnique Cluster: C = Clustered Or N = Non-Clustered
Views	
AutoQuote[Entity]	Views used for the AutoQuote Proces. Use Camel Case
MNet[Entity]	
SpecialDiscount[Entity]	
Stored Procedures	TBD
Functions	
	TBD

Naming Conventions - Data Import

Environments	Alias	
devdw3.maintenancenet.net\dw_stage	DWSERVER	Development
		QA
		Stage
dw3.maintenancenet.net	DWSERVER	Production

Databases	Definition
Campaign	This is used to preprocessing data that will be moved into the Campaign database in AppServer and used by Campaign Manager. Currently the Campaign Manager is used for welcome campaigns, registration and certificates.
DataCore	Normalized imported data. The main import processes/transforms raw data feeds and stores the information in this database. This also contains the main routines used by the 'Batch Update' process.
ETL_Logging	This contains the logging information for specific processes running (imports, maintenance plans, batch update, etc)
MNetImport	This contains the main import routines that processes raw data feeds and imports them into the DataCore.
MNetRaw	Stores all the raw data from the clients files
MNetReference	Stores catalog data from the manufacturer, parameter tables, and configuration tables
MNetReport	This is the legacy database that was used by the batch update. Currently it still contains reporting routines.
MNetSource	This is the legacy database that stored the imported raw feeds in a normalized format. This will be decommissioned once all processes are migrated to use the new ETL process. This is replaced by the DataCORE.
Project	Is for store bkup data before executing deletions
ReportServer	Stores the data for SSRS
ReportServerTempDB	?

Naming Conventions	
Schemas	
Dbo = default	Do not use any other schema names in this db.
Tables	
tbl_[Entity][SubEntity]	"tbl_" used for all tables, use plural not singular names for entities, camel case only.
Example: tbl_ContactsLocations	

For MnetRaw db	
If the data is being imported from a client using the following naming convention	
For Raw Files tbl[Manufacturer]_[Customer]_[STG]_[Import Type]_[File Type] For All Raw Files (where all raw files with same format consolidated) tbl[Manufacturer]_[Customer]_[STG]_[ALL]_[Import Type]_[File Type]	Here are some examples tblCisco_Disti_STG_DOT_ActiveExpiring tblCisco_Disti_STG_DOT_Expired tblCisco_Disti_STG_ALL_DOT tblCisco_CDW_STG_PMC_DirectATT tblCisco_CDW_STG_PMC_InDirectATT tblCisco_CDW_STG_PMC_InDirectATTLLW tblCisco_CDW_STG_ALL_PMC
For example:	
Manufacturer = Cisco	Cisco, Mfg (all Manufacturers)
Customer = CDW	This represents the specific customer the data feed is for. For Example - CDW, IngramUS, IngramCA, Westcon, Azlan, Disti (all distributors)
Import Type = PMC / CSD	Other Types:PMC, CSD, DOT, LTNT, SNIF
File Type = DirectATT, HWShip, Expired, etc.	File Type: Usally the name of the file excluding any dates in the file name.
Table Name = tblCisco_CDW_STG_PMC_DirectATT	
For ETL_Logging	
maint_[MainProcess]_[Subprocess1][Subprocess2]…	Specific to maintenance plans Example: maint_Indexes_Rebuild_ReOrg
LoadLog_[MainProcess]_[Subprocess1][Subprocess2]…	Specific to logging tables Example: LoadLog_RejectedRows
For MNetImport	
tblAuto_[Process]	Automain specific tables.
For Example:	
tblAuto_DataFeed	

Indexes	
Ix_[Column1][Column2][Column3][Column4]_[Type][Cluster]	Use the column names in the index if there is more than four just use the first four columns. Type: U = Unique Or N = NonUnique Cluster: C = Clustered Or N = Non-Clustered
For Example:	
ix_ContactID_UC	
ix_ContactIDLastNameEmail_NN	
Views	
v_	"v_" used for all views, use plural not singular names for entities, camel case only.
Stored Procedures	
prc_[MainProcess]_[Subprocess1][Subprocess2]…	For all other stored procedures. These are some subprocess examples Extract = Term used to extract data from the raw staging tables Update = Term to update data Delete = Term used to delete records Insert = Term used to insert data Create = Term used to create an object Xform = Term used to transform or map data from raw to generic columns. Aug = Term use to Augment data from other tables Apply = Term used to apply business/uncertified rules Get = used to Get data for processing purposes
mi_[MainProcess]_[Subprocess1][Subprocess2]…	Main Import Procedures Specfic. These should be compiled in the MNetImport database.
bu_[MainProcess]_[Subprocess1][Subprocess2]…	Batch Update Procedures Specfic. These should be complied in the DataCORE database.

prc_[Manufacturer]_[Customer]_extract_[Import Type]_[File Type]	Specific to process used to load raw data files into staging tables. These should be complied in the MnetRaw database. Examples: prc_Cisco_Disti_Extract_DOT_ActiveExpiring prc_Cisco_Disti_Extract_DOT_Eliminated prc_Cisco_Disti_Extract_LTNT_ActiveExpiring prc_Cisco_Disti_Extract_LTNT_Expired
prc_[Manufacturer]_[Customer]_xform_tblInventoryImports_DataTypes_[Import Type]	Specific to process used to load map raw data tables to standard import table. These should be complied in the MnetRaw database. prc_Cisco_Disti_xform_tblInventoryImports_DataTypes_DOT prc_Cisco_Disti_xform_tblinventoryImports_DataTypes_LTNT
Functions	
Fn_	
Triggers	
(table name)_ChangeLog	Specific to logging tables found in the DataCore database Example: tblAssets_ChangeLog

Naming Conventions - Unified Portal

Environments	Alias	
devdb5.maintenancenet.net\dev		development
devdb5.maintenancenet.net\qa		QA
devdb5.maintenancenet.net\stage		Stage
prodappdb.maintenancenet.net	APPSERVER	Production

Databases	Definitions
ASPState	
Campaign	
CiscoBuyNow	
Logging	
MNetReference	Stores interal reference & configuration tables
MNetReport	
MNetReportUP	Stores the data for the Unified Portal (Service360)
MNetRS	
MNetStore	
Quotes	This stores all the information regarding quotes for Auto quoting and quoting in the UP
Rules	
Service360	
ServiceCore	
test	
WFInstanceStore	

Naming Conventions	
Schemas	Definitions
Tables	
Tbl_	TBD
Indexes	
Ix_	TBD
Views	
V_	TBD

Stored Procedures	
Prc_	TBD
Functions	
Fn_	TBD

SQL Database Design Best Practices

SQL Database Design Best Practices

Follow the SQL Standards for naming conventions; if you are unsure ask your technical lead.

Basics

Ensure the process you are developing is generic as possible

Comments are easier to read than Code

Stored Procedures

Stored Procedures

Optimize your Stored Procedures

Avoid Hardcoding Values

Drop Your Temp Tables

Aliases used for fields

Format your code

Tables

Tables

Indexing

Deleting Data

Databases

Creating Databases

Linked Servers

1 Basics

Ensure the process you are developing is generic as possible

When developing a process avoid putting the client name in the process unless the data is a data import and will only hold that clients data. Other than that always think that a process could be used by multiple clients/users/developers even though it has been requested by just one client/user and design it in such a way that it can be used by multiple clients/users/developers. So instead of client name in the object put a field with the client name or ID instead and join to the relevant table. Also name the objects more about what it actually does like update, augment, get, etc.

Comments are easier to read than Code

Write comments in your stored procedures, triggers and SQL batches generously, especially when something is not very obvious or it's a complicated business rule. This helps other programmers understand your code clearly and avoids them asking you what it means. Don’t worry about the length of the comments, as it won’t impact the performance, unlike interpreted languages like ASP 2.0. Put in the header tags links to any supporting documentation. Always think of some else reviewing this for the first time and what information they would need to modify this without having to ask.

If you are unsure ask

If you are unsure about any of the best practices or naming conventions then ask. Your code may get rejected for using bad practices and will need to be rewritten. Also if can think of any best practices that you fell should be included let your Team Lead know.

Happy coding =)

2 Stored Procedures

Store Procedures
Add a header tag as shown below. Make sure if you have edited it explain the reason why you are making the edits and the ticket number as well if applicable when you check it in to redgate SVN.

Do not use profiler against production unless there is no other option and if you do be sure to keep it brief and stop it.

Place all declare statements before any other code in the procedure to give the query optimizer the best shot at reusing query plans.

Creating a Stored Procedure

1. All stored procedures will start with a USE stmt followed by the db name
2. Next, SPS must have an IF..EXISTS..DROP PROCEDURE statement.
3. Next, the SP should have a brief description of what the SP does.
4. Next, the CREATE statement. Never use ALTER.
5. All variables should be declared at the top just after the create statement.
6. All SPs must be checked into SVN.

Go to stored procedure template to see an example.

Editing Stored Procedures
Modifications of data must be logged. Any time the data changes, there must be a record, especially when modifying customer data.

Put in the comments header the date and the reason why you made the edit add a ticket reference number if possible.

Make sure you use 'Format SQL' to clean up your code and keep it in a constant format.

2 a) Optimize your Stored Procedures

Once you are getting the results you need from your Stored Procedure and it is a stored procedure that will be used frequently then spend a few hours optimizing your script.

- Don't use SELECT * only select the fields you need the less the better.
- Do not use NO LOCK hints under any circumstances, use the optimizer instead.
- Use the execution plan to identify the slowest performance in your script.
- Join on indexed integer fields wherever possible. Avoid text values in queries and wildcard searches when possible.

Sometimes complex joining and where clauses can be very slow in one go. Sometimes it is better to create a temp tables and breaking up the code into a series of smaller steps than to try and get it all in one step.

Try to avoid wildcard characters at the beginning of a word while searching using the LIKE keyword, as that results in an index scan, which defeats the purpose of an index. The following statement results in an index scan, while the second statement results in an index seek:

```
SELECT LocationID
FROM Locations
WHERE Specialities LIKE '%pples'

SELECT LocationID
FROM Locations
WHERE Specialities LIKE 'A%s'
```

Also avoid searching using not equals operators (<> and NOT) as they result in table and index scans.

Parameter Sniffing

Process by which SQL Server creates an optimal execution plan for a stored procedure by using the parameters that are passed the first time a stored procedure is executed. Subsequent execution of the same stored procedure may yield varying turnaround times if different parameter values are used).

To work around this, create a local variable(s) within the stored procedure to hold the parameter value(s). Use these variables in place of the parameters throughout the stored procedure. The local variable should be named @{name}Local.

Example:

```
CREATE PROCEDURE [Report].[prc_getAutoQuoteMetricsDetails] @SnapshotID INT

AS

SET NOCOUNT ON


BEGIN



DECLARE @SnapshotIDLocal INT

DECLARE @SoldToParty INT

DECLARE @ReportEvenWhenEmailMissing BIT



SET @SnapshotIDLocal = @SnapshotID



SELECT  @SoldToParty = A.SoldToParty ,   @ReportEvenWhenEmailMissing =
ISNULL(ReportEvenWhenEmailMissing, 0)

FROM    Snapshot.autoquoteconfig A

JOIN Snapshot.AutoQuoteBatch B ON A.SnapshotId = B.SnapshotId AND A.AutoQuoteConfigID =
B.AutoQuoteConfigID

WHERE   A.SnapshotID = @SnapshotIDLocal
```

2 b) Avoid Hardcoding Values

Sometimes this can be tricky but avoid hardcoding values in your code as the staging and production environments are not in sync and use different ID's.
Also If additional status ID's need to be included then all code written with that ID will need to be recoded.

It is better to pass them as a parameter using the parameter tables in MNetReference database.

2 c) Drop Your Temp Tables

If you are using temp tables in your stored procedure always add the following script before creating a temp table.

IF OBJECT_ID('tempdb..#TEMPNAME') IS NOT NULL DROP TABLE #TEMPNAME

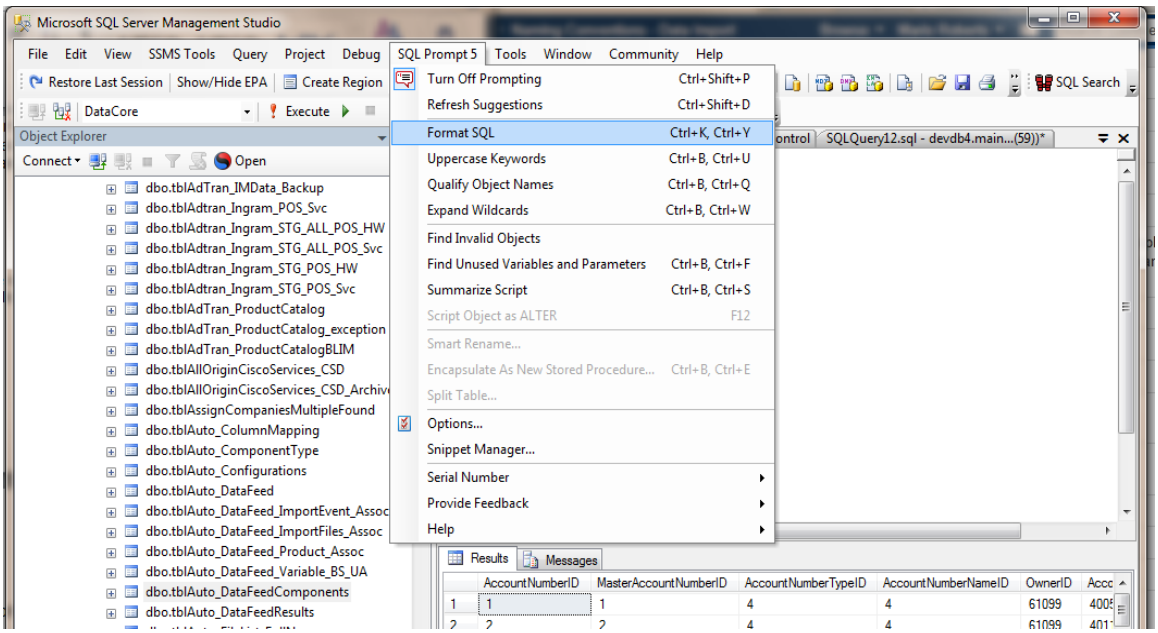
This helps will developing, debugging and testing. Also at the very end of your Stored Procedure add the same code, so that no temp tables are left in tempdb.

2 d) Aliases used for fields

If you are joining to multiple tables and the select statement you are creating is pulling fields from multiple tables. Then use the table alias names to help identify which field came from what table. This helps when you revisit code months later and also helps other coders understand the code also.

2 e) Format your code

In some cases formatting your code will highlight the problem that you're trying to debug. It also helps being able to read the code faster when it is formatted correctly. There is a tool that can help format the code quickly go to QL Prompt > Format SQL.



2 f) Avoid using Cursors

Try to avoid server side cursors as much as possible. Always stick to a 'set-based approach' instead of a 'procedural approach' for accessing and manipulating data. Cursors can often be avoided by using SELECT statements instead.

If a cursor is unavoidable, use a WHILE loop instead. I have personally tested and concluded that a WHILE loop is

always faster than a cursor. But for a WHILE loop to replace a cursor you need a column (primary key or unique key) to identify each row uniquely. I personally believe every table must have a primary or unique key.

Example:

```
WHILE ( SELECT  AVG(ListPrice)
        FROM    Production.Product
      ) < $300
   BEGIN
        UPDATE  Production.Product
        SET     ListPrice = ListPrice * 2
        SELECT  MAX(ListPrice)
        FROM    Production.Product
        IF ( SELECT MAX(ListPrice)
             FROM   Production.Product
           ) > $500
           BREAK
        ELSE
           CONTINUE
   END
PRINT 'Too much for the market to bear'
```

2 g) Stored Procedure Template

```
USE [DATABASE_NAME]
GO

/****** Object:  StoredProcedure [dbo].[prc_Template]    Script Date: 04/25/2012 15:44:25 ******/
IF  EXISTS (SELECT * FROM sys.objects WHERE object_id = OBJECT_ID(N'[dbo].[prc_Template]') AND type in (N'P', N'PC'))
```

```
DROP PROCEDURE [dbo].[prc_Template]

GO


/****** Object:  StoredProcedure [dbo].[prc_Template]    Script Date: 04/25/2012 15:44:25 ******/

SET ANSI_NULLS ON
GO

SET QUOTED_IDENTIFIER ON
GO


 /*****************************************************************************
        Desc: ENTER THE DESCRIPTION AND PURPOSE OF THE STORED PROCEDURE
              INCLUDE URL TO SUPPORTING DOCUMENTATION

**
**        Called by:  ENTER SP's, SSIS etc.
**
**        Parameters:
**        Input             Description
**     ----------------------------------------------------------------
**        @PARAM1 INT = NULL     ENTER YOUR DESCRIPTION OF THE PARAMETER
          @Debug BIT = 0          Whether to go into Debug mode or not. In Debug mode, a temp table is created.
**
**        Output            Description
**     ----------------------------------------------------------------
**
**        Auth: YOUR NAME
**        Date: DD/MM/YYYY

*****************************************************************************
**        Change History
*****************************************************************************
**        Date:              Author:                 Description:
**        --------           --------                --------------------------------
**        DD/MM/YYYY  MR EXAMPLE          DESCRIBE YOU CHANGE INCLUDE TICKET NUMBER IF IT
HELPS
**        DD/MM/YYYY  MR EXAMPLE          DESCRIBE YOU CHANGE INCLUDE TICKET NUMBER IF IT
HELPS

*****************************************************************************/


CREATE PROCEDURE [dbo].[prc_Template]

AS

BEGIN
```

```
    SET NOCOUNT ON;

IF OBJECT_ID('tempdb..#TEMPNAME') IS NOT NULL DROP TABLE #TEMPNAME

SELECT [NAME]
FROM tbl_Table
INTO #TEMPNAME
WHERE PARAM1 = @PARAM1

SELECT [NAME]
FROM #TEMPNAME

IF OBJECT_ID('tempdb..#TEMPNAME') IS NOT NULL DROP TABLE #TEMPNAME

END

GO
```

2 h) Stored Procedure Recompilation

Recompilations Due to Interleaving Data Definition Language (DDL) and Data Manipulation Language (DML) Operations

If DDL operations are performed within a procedure or batch, the procedure or batch is recompiled when it encounters the first subsequent DML operation affecting the table involved in the DDL.

Consider the following example stored procedure:

```
drop procedure Interleave
go
create procedure Interleave as
-- DDL
create table t1 (a int)
-- DML
```

```
select * from t1

-- DDL

create index idx_t1 on t1(a)

-- DML

select * from t1

-- DDL

create table t2 (a int)

-- DML

select * from t2

go

exec Interleave
```

If you execute this code in Query Analyzer and view the above events in a Profiler trace, you will see the following sequence:

Collapse this tableExpand this table

Event Class	Text
SP:Starting	Interleave
SP:StmtStarting	create table t1 (a int)
SP:StmtStarting	select * from t1
SP:Recompile	Interleave
SP:StmtStarting	select * from t1
SP:StmtStarting	create index idx_t1 on t1(a)
SP:StmtStarting	select * from t1
SP:Recompile	Interleave
SP:StmtStarting	select * from t1
SP:StmtStarting	create table t2 (a int)

SP:StmtStarting	select * from t2
SP:Recompile	Interleave
SP:StmtStarting	select * from t2
SP:Completed	Interleave

In this case, the stored procedure is recompiled three times during execution. To understand why this happens, consider how the optimizer develops a plan for this stored procedure:

During initial compilation of the procedure, the tables t1 and t2 do not exist. Therefore, no plan for the queries referencing these tables can be created. They must be generated at execution time.

As the procedure executes for the first time, the first step is to create table t1. The next step is a select from table t1 -- which there is no plan for. Therefore, the procedure is recompiled at this point to develop a plan for the SELECT statement. A plan is generated for the current select from t1, as well as the select from t1 after the index creation. No plan can be generated for the select from t2 because t2 still does not exist yet.

The next step is to create an index on t1. Following that, another select is performed on t1, which now has a plan from the first recompile. However, because the schema of t1 has been changed since that plan was generated, the procedure must be recompiled again to generate a new plan for the select from t1. And because t2 still does not exist, no plan can be generated for the select from t.

Next, the table t2 is created and the select from t2 is executed. Because no plan exists for statement, the procedure is recompiled a final time.

These recompilations occur on every execution of the stored procedure. To reduce the recompilations, modify the procedure to do all DDL operations first, followed by DML operations, as shown in the following:

```
drop procedure NoInterleave

go

create procedure NoInterleave as

-- All DDL first

create table t1 (a int)

create index idx_t1 on t1(a)

create table t2 (a int)

-- Then DML

select * from t1

select * from t1

select * from t2

go
```

exec NoInterleave

exec NoInterleave

The first execution of the NoInterleave procedure will show the following events in Profiler:

Collapse this tableExpand this table

Event Class	Text
SP:Starting	NoInterleave
SP:StmtStarting	create table t1 (a int)
SP:StmtStarting	create index idx_t1 on t1(a)
SP:StmtStarting	create table t2 (a int)
SP:StmtStarting	select * from t1
SP:Recompile	NoInterleave
SP:StmtStarting	select * from t1
SP:StmtStarting	select * from t1
SP:StmtStarting	select * from t2
SP:Completed	NoInterleave

In this case all of the DDL statements are done up front. The optimizer compiles this procedure as follows:

During initial compilation of the procedure, the tables t1 and t2 do not exist. Therefore, no plan for the queries referencing these tables can be created. They must be generated at execution time.

The first steps the procedure performs are the DDL operations, creating tables t1 and t2, as well as the index on t1.

The next step is the first select from t1. Because there is no plan available for this SELECT statement, the procedure is recompiled. Because all of the objects exist, plans are generated for all of the SELECT statements in the procedure at this time.

The rest of the procedure executes using the plans generated. Because there are no changes to the referenced objects, there is no need to recompile the procedure further.

NOTE: The second and subsequent executions make use of the existing query plan and cache, and do not result in any recompilations at all. Procedures that create, alter, or drop tables should be modified to ensure that all DDL statements are located at the beginning of the procedure.

http://support.microsoft.com/kb/243586

3 Tables

Creating Tables
All tables must have a primary key with the column name being the table name plus “ID” at the end. For example table name tblContacts the primary key is contactsID. Also Every table must have a clustered index see indexes for more information.

For table naming conventions click here

Table Name Aliases

If you are using aliases in your script make the alias name using the first letters of each word in the table name for example tblMasterProductCatalog as MPC. You will notice that the aliases in most scripts will have the same or simular naming convention which reduces code noise.

3 a) Indexing

All tables are required to have a clustered index. If the performance of the queries/stored procedures is still slow test all stored procedures associated to that table/s. Review any common fields they are filtered or join to. If they are not in your clustered index, then create additional index/es as applicable then retest your store procedures for performance.

For Index naming conventions click here

3 b) Deleting Data

Data should never be deleted. If data is no longer to be used, flag data as inactive, and/or archive data. If those options are not available, escalate accordingly and acquire proper approval before proceeding.

If you do acquire approval to delete data, back up the data first even it’s been checked multiple times. Insert the data you are planning to delete to the project database before actually making the deletion from production databases.

4 Databases

4 a) Creating Databases

Speak with your Team Lead first before creating a new database. We may find that we can house the tables/view/store procedures etc. in an existing database.

4 b) Linked Servers

When using SPs, functions, SSRS and SSIS, refer to the application server as APPSERVER in linked server calls and connection strings in both DEV and PROD, never refer to it by the server name.

Within an SP, do not reference the server the SP resides on. Only reference the server for external calls. For

example, on ETL server, you would not call DWserver.mnetraw.dbo.tInventoryImport. Instead you would drop the DWserver unless the Sp resided on APPSERVER.

Do not create linked servers. Use what is provided. If a new one is required, send request to the DBA.

5 Quality Assurance

1. Check SP's are in devdw2 and are stored in the following location: https://svn.maintenancenet.net/core/DataServices/Core/trunk/StoredProcedures and Functions
2. If an SSIS package is created make sure it exists in https://svn.maintenancenet.net/core/DataServices/Core/trunk/SSIS in the correct project folder.
3. Read the requirements on the ticket and check that the process has fulfilled those requirements.
4. Check the mapping with the current documentation.
5. Check ID's that are hardcoded and referenced correctly i.e. manufacturerID = 76 should be Cisco
6. Run it in dev environments and check the results at the end of the process.
7. Produce an excel list of results for the BA to analyze and approved if possible.
8. Check and make sure that our standards and best practices have been applied.
9. If it's an edit to a SP or function make sure the developer has written a comment in the header tag mentioning the edit/s they have done and reference a ticket if it's complicated.